



Apuntes Ansible: De Cero a Master

Guía profesional, práctica y escalable

Objetivo: dominar Ansible en escenarios reales: Linux, Windows, red, on-premise, cloud, Kubernetes, seguridad, CI/CD y operación enterprise.

DevOps Notes

17 de marzo de 2026

Índice

1. Cómo usar estos apuntes (ruta recomendada)	2
1.1. Ruta por niveles	2
2. Qué es Ansible y por qué importa	2
2.1. Problemas que resuelve	2
2.2. Conceptos base	2
3. Ansible en on-premise, cloud e híbrido	2
3.1. Targets típicos	3
3.2. Patrón híbrido recomendado	3
4. Instalación, estructura y ciclo de trabajo	3
4.1. Instalación recomendada	3
4.2. Flujo básico	3
4.3. Estructura enterprise recomendada	4
5. Inventario en profundidad	4
5.1. Inventario estático INI	4
5.2. Inventario YAML	4
5.3. Inventario dinámico	5
6. Ad-hoc commands y primeras automatizaciones	5
7. Playbooks y idempotencia profesional	5
7.1. Playbook base	5
7.2. Idempotencia real	5
8. Variables, precedence y modelado limpio	6
8.1. Fuentes comunes de variables	6
8.2. Reglas prácticas	6
9. Roles y colecciones: diseño enterprise	6
9.1. Estructura de role	6
9.2. Colecciones y dependencias	6
10. Templates, handlers y control de cambios	7
10.1. Buenas prácticas	7
11. Control de flujo avanzado	7
11.1. when, loop, tags, include/import	7
11.2. Manejo robusto de errores	7
12. Facts, descubrimiento y custom facts	8
12.1. Uso de facts	8
13. Ansible Vault y gestión de secretos	8
13.1. Comandos esenciales	8
13.2. Patrón seguro de secretos	8

14.Kubernetes con Ansible (EKS/AKS/GKE/on-prem)	9
14.1. Módulos y enfoque recomendado	9
14.2. Patrón Terraform + Ansible + GitOps	9
15.Windows, red y automatización heterogénea	9
15.1. Windows con WinRM	9
15.2. Networking automation	9
16.Rendimiento y estrategia de ejecución	9
16.1. Parámetros clave	9
17.Testing, calidad y CI/CD	10
17.1. Cadena mínima de calidad	10
17.2. Molecule para roles	10
18.AWX / Ansible Automation Platform	10
18.1. Qué aporta a escala	10
18.2. Modelo de operación recomendado	11
19.Seguridad y compliance	11
20.Migración desde scripts legacy a Ansible	11
20.1. Plan de migración por fases	11
20.2. Matriz de riesgo	11
21.Anti-patrones que debes evitar	11
22.Troubleshooting experto	12
22.1. Errores comunes y solución	12
22.2. Comandos de depuración	12
23.Checklist de salida a producción (Go-Live)	12
24.Laboratorios guiados (de básico a experto)	12
25.Roadmap de estudio: de 0 a master	13
26.Cheat Sheet final	13
27.Glosario rápido	14
28.Conclusión	14

1 Cómo usar estos apuntes (ruta recomendada)

Método de aprendizaje: 30 % teoría + 70 % laboratorio. Ejecuta cada ejemplo en un entorno de práctica antes de llevarlo a producción.

1.1 Ruta por niveles

Nivel	Objetivo práctico
Inicial	Inventarios, comandos ad-hoc, playbooks simples e idempotencia.
Intermedio	Roles, variables por entorno, Vault, handlers y templates.
Avanzado	CI/CD, testing, seguridad, automatización híbrida y day-2 operations.
Master	Plataforma automatizada con gobernanza, AWX/AAP, runbooks y compliance.

2 Qué es Ansible y por qué importa

Ansible es una plataforma de automatización **agentless** orientada a configuración, despliegue, orquestación y operación repetible.

2.1 Problemas que resuelve

- Eliminación de tareas manuales repetitivas e inconsistentes.
- Estandarización de configuración entre entornos.
- Reducción de drift operacional y errores humanos.
- Trazabilidad de cambios vía Git + pipelines.

2.2 Conceptos base

- **Inventory:** hosts y grupos objetivo.
- **Playbook:** automatización declarativa en YAML.
- **Task:** unidad mínima de acción.
- **Role:** empaquetado reusable de automatización.
- **Collection:** distribución modular de plugins/módulos.
- **Vault:** cifrado de secretos.

3 Ansible en on-premise, cloud e híbrido

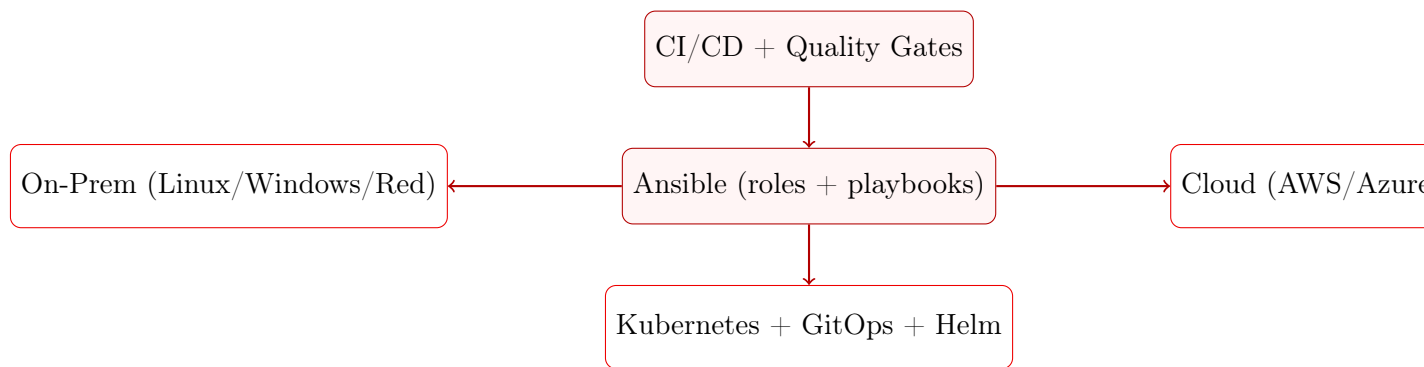
Sí, Ansible funciona perfectamente en **on-premise**. De hecho, es uno de sus casos más fuertes.

3.1 Targets típicos

- Linux (Ubuntu, RHEL, Debian, Rocky, etc.).
- Windows Server (WinRM).
- Equipos de red (Cisco, Juniper, Arista, Fortinet, Palo Alto).
- Virtualización (VMware), cloud pública y Kubernetes.

3.2 Patrón híbrido recomendado

1. Terraform aprovisiona infraestructura base.
2. Ansible configura sistemas y operación day-2.
3. GitOps gestiona aplicaciones en Kubernetes.



4 Instalación, estructura y ciclo de trabajo

4.1 Instalación recomendada

```
python3 -m pip install --user pipx
python3 -m pipx ensurepath
pipx install ansible
ansible --version
```

Listing 1: Instalación con pipx

4.2 Flujo básico

```
ansible-inventory -i inventories/dev/hosts.ini --graph
ansible all -i inventories/dev/hosts.ini -m ping
ansible-playbook -i inventories/dev/hosts.ini playbooks/site.yml
ansible-playbook -i inventories/dev/hosts.ini playbooks/site.yml --
  check --diff
```

Listing 2: Comandos esenciales

4.3 Estructura enterprise recomendada

```
ansible-platform/  
|- -- ansible.cfg  
|- -- collections/  
|- -- inventories/  
|   |- -- dev/  
|   |- -- qa/  
|   +- -- prod/  
|- -- playbooks/  
|   |- -- site.yml  
|   |- -- bootstrap.yml  
|   |- -- patching.yml  
|   +- -- dr-recovery.yml  
|- -- roles/  
|- -- molecule/  
|- -- docs/  
+- -- .github/workflows/
```

Listing 3: Blueprint de repositorio Ansible

5 Inventario en profundidad

5.1 Inventario estático INI

```
[web]  
web01 ansible_host=10.10.1.11 ansible_user=ubuntu  
web02 ansible_host=10.10.1.12 ansible_user=ubuntu  
  
[db]  
db01 ansible_host=10.10.2.21 ansible_user=ubuntu  
  
[all:vars]  
ansible_python_interpreter=/usr/bin/python3
```

Listing 4: hosts.ini

5.2 Inventario YAML

```
all:  
  children:  
    web:  
      hosts:  
        web01:  
          ansible_host: 10.10.1.11  
        web02:  
          ansible_host: 10.10.1.12  
    db:  
      hosts:  
        db01:
```

```
ansible_host: 10.10.2.21
```

Listing 5: hosts.yml

5.3 Inventario dinámico

En cloud, usa inventory dinámico para evitar listas manuales. Ejemplos: `amazon.aws.aws_ec2`, `azure.azure_rm`, `google.cloud.gcp_compute`.

6 Ad-hoc commands y primeras automatizaciones

```
ansible all -i inventories/dev/hosts.ini -m ping
ansible web -i inventories/dev/hosts.ini -m shell -a "uptime"
ansible web -i inventories/dev/hosts.ini -m apt -a "name=nginx state
    =present" -b
ansible all -i inventories/dev/hosts.ini -m setup
```

Listing 6: Comandos ad-hoc clave

7 Playbooks y idempotencia profesional

7.1 Playbook base

```
- name: Configurar web servers
  hosts: web
  become: true
  tasks:
    - name: Instalar nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Habilitar servicio nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

Listing 7: Instalación y servicio

7.2 Idempotencia real

- Prefiere módulos declarativos sobre `shell/command`.
- Define estados explícitos (`present`, `absent`, `started`).
- Evita tareas no deterministas.

8 Variables, precedence y modelado limpio

8.1 Fuentes comunes de variables

- defaults/main.yml (role defaults).
- group_vars/host_vars.
- Variables en playbook/role.
- Extra vars (-e) con prioridad alta.

8.2 Reglas prácticas

1. Valores por defecto en defaults.
2. Valores por entorno en group_vars.
3. Nunca esconder lógica crítica solo en -e.

9 Roles y colecciones: diseño enterprise

9.1 Estructura de role

```
roles/  
+- -- nginx/  
    |- -- defaults/main.yml  
    |- -- vars/main.yml  
    |- -- tasks/main.yml  
    |- -- handlers/main.yml  
    |- -- templates/nginx.conf.j2  
    |- -- files/  
+- -- meta/main.yml
```

Listing 8: Role reusable

9.2 Colecciones y dependencias

```
collections:  
- name: ansible.posix  
- name: kubernetes.core  
- name: community.general  
- name: amazon.aws
```

Listing 9: requirements.yml

```
ansible-galaxy collection install -r requirements.yml
```

Listing 10: Instalar colecciones

10 Templates, handlers y control de cambios

```
- name: Render nginx config
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    mode: '0644'
  notify: restart nginx

handlers:
- name: restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

Listing 11: Template + notify handler

10.1 Buenas prácticas

- Cambios config siempre con `notify`.
- Handlers idempotentes y específicos.
- Plantillas versionadas y testeadas.

11 Control de flujo avanzado

11.1 `when`, `loop`, `tags`, `include/import`

```
- name: Instalar paquetes base
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop:
    - curl
    - git
    - jq
  tags: [base, packages]

- name: Ejecutar sólo en Debian
  ansible.builtin.service:
    name: cron
    state: restarted
  when: ansible_facts['os_family'] == 'Debian'
```

Listing 12: Flujo avanzado

11.2 Manejo robusto de errores

```
- name: Cambio crítico con rollback
  block:
    - name: Aplicar cambio
      ansible.builtin.command: /usr/local/bin/apply_change

  rescue:
    - name: Ejecutar rollback
      ansible.builtin.command: /usr/local/bin/rollback_change

  always:
    - name: Registrar estado
      ansible.builtin.debug:
        msg: "Cambio finalizado"
```

Listing 13: block/rescue/always

12 Facts, descubrimiento y custom facts

12.1 Uso de facts

- Descubrir SO, red, memoria, discos, interfaces, etc.
- Adaptar tareas por contexto del host.

```
- name: Mostrar familia de SO
  ansible.builtin.debug:
    msg: "{{ inventory_hostname }}" -> "{{ ansible_facts['os_family'] }}"
```

Listing 14: Ejemplo con facts

13 Ansible Vault y gestión de secretos

13.1 Comandos esenciales

```
ansible-vault create group_vars/prod/vault.yml
ansible-vault edit group_vars/prod/vault.yml
ansible-vault view group_vars/prod/vault.yml
ansible-playbook -i inventories/prod/hosts.ini playbooks/site.yml --ask-vault-pass
```

Listing 15: Vault básico

13.2 Patrón seguro de secretos

- Separar secretos por entorno y dominio.
- Usar vault IDs y rotación regular.
- Integrar con gestores externos (Vault/Secrets Manager).

14 Kubernetes con Ansible (EKS/AKS/GKE/on-prem)

14.1 Módulos y enfoque recomendado

- Colección `kubernetes.core` para recursos K8s.
- Helm vía módulos o CLI controlada.
- Ansible para bootstrap y operación; GitOps para apps de alto churn.

```
- name: Operación sobre k8s
hosts: localhost
gather_facts: false
tasks:
  - name: Crear namespace
    kubernetes.core.k8s:
      state: present
      definition:
        apiVersion: v1
        kind: Namespace
        metadata:
          name: plataforma
```

Listing 16: Namespace + deployment simple

14.2 Patrón Terraform + Ansible + GitOps

1. Terraform crea clúster y red base.
2. Ansible instala add-ons operativos (monitoring/logging/seguridad).
3. GitOps gestiona aplicaciones y releases continuas.

15 Windows, red y automatización heterogénea

15.1 Windows con WinRM

- Usa `ansible.windows` y `community.windows`.
- Asegura certificados, TLS y privilegios mínimos.

15.2 Networking automation

- Backups de configuración, validaciones y despliegues controlados.
- Ejecución por lotes y validación previa en entorno staging.

16 Rendimiento y estrategia de ejecución

16.1 Parámetros clave

- `forks`: paralelismo.

- `serial`: rolling updates.
- `strategy`: `linear` / `free`.
- `throttle`: limitar concurrencia en tareas sensibles.

```
- name: Deploy por lotes
hosts: web
serial: 2
become: true
tasks:
  - name: Actualizar app
    ansible.builtin.apt:
      name: myapp
      state: latest
```

Listing 17: Rolling update seguro

17 Testing, calidad y CI/CD

17.1 Cadena mínima de calidad

```
yamllint .
ansible-lint
ansible-playbook -i inventories/dev/hosts.ini playbooks/site.yml --
  syntax-check
ansible-playbook -i inventories/dev/hosts.ini playbooks/site.yml --
  check --diff
```

Listing 18: Pipeline recomendada

17.2 Molecule para roles

```
pipx install molecule
cd roles/nginx
molecule init scenario --driver-name docker
molecule test
```

Listing 19: Test de role con Molecule

18 AWX / Ansible Automation Platform

18.1 Qué aporta a escala

- RBAC, auditoría y ejecución centralizada.
- Job templates, schedules, credenciales seguras.
- Integración con Git, inventarios dinámicos y approvals.

18.2 Modelo de operación recomendado

1. Proyecto en Git con PR obligatorias.
2. AWX/AAP ejecuta playbooks versionados.
3. Evidencia de ejecución almacenada para compliance.

19 Seguridad y compliance

- Principio de mínimo privilegio (SSH/WinRM/sudo).
- Secretos solo por Vault o gestor externo.
- Inventarios, playbooks y cambios siempre auditables.
- Hardening automatizado (benchmarks CIS) y verificación continua.

20 Migración desde scripts legacy a Ansible

20.1 Plan de migración por fases

1. Inventariar scripts críticos y dependencias.
2. Convertir tareas comunes a roles reutilizables.
3. Añadir tests y validación en CI.
4. Decomisionar scripts antiguos por etapas.

20.2 Matriz de riesgo

Tipo de cambio	Riesgo	Mitigación
Cambios de red/firewall	Alto	Ventana controlada, validación previa, rollback probado.
Parches SO en masa	Medio-Alto	serial , canary y monitoreo post-cambio.
Deploy de app stateless	Medio	Blue/green o rolling con health checks.
Hardening baseline	Medio	Entorno piloto + escaneo de cumplimiento.

21 Anti-patrones que debes evitar

1. Playbooks monolíticos gigantes sin roles.
2. Uso excesivo de `shell/command` para todo.
3. Variables opacas y sin convención de nombres.
4. Secretos en texto plano en repositorio.
5. Ejecutar en producción sin `-check` previo en entorno similar.

22 Troubleshooting experto

22.1 Errores comunes y solución

Problema	Acción recomendada
UNREACHABLE por SSH	Revisar red, usuario, llaves, firewall, <code>ansible_host</code> .
Cambios en cada ejecución	Revisar idempotencia y módulos no declarativos.
Error de permisos sudo	Validar <code>become</code> , <code>become_user</code> y <code>sudoers</code> .
Variables no aplican	Revisar precedencia y colisiones <code>group_vars</code> / <code>host_vars</code> .
Timeouts frecuentes	Ajustar <code>forks</code> / <code>serial</code> / <code>timeout</code> y dividir tareas pesadas.

22.2 Comandos de depuración

```
ansible-inventory -i inventories/dev/hosts.ini --graph
ansible all -i inventories/dev/hosts.ini -m setup
ansible-playbook -i inventories/dev/hosts.ini playbooks/site.yml -vvv
```

Listing 20: Debug rápido

23 Checklist de salida a producción (Go-Live)

Checklist de producción

- Inventarios versionados y segregados por entorno.
- Secrets cifrados con Vault y rotación validada.
- `yamllint`, `ansible-lint`, `-check -diff` obligatorios en PR.
- Runbook de rollback documentado y probado.
- Despliegue gradual por `serial` para servicios críticos.
- Trazabilidad completa de ejecución (job IDs, logs, aprobaciones).

24 Laboratorios guiados (de básico a experto)

Lab 1: Fundamentos

Inventario + conectividad + playbook base sobre 3 VMs Linux.

Lab 2: Roles y templates

Crear role reutilizable de NGINX con handlers, variables y validaciones.

Lab 3: Seguridad

Introducir Vault, separación dev/prod y políticas mínimas de acceso.

Lab 4: CI/CD y testing

Integrar yamllint + ansible-lint + check mode + Molecule.

Lab 5: Híbrido real

Automatizar Linux + Windows + namespace k8s con pipeline controlada.

25 Roadmap de estudio: de 0 a master

Fase 1 (1-2 semanas)

- Inventario, ad-hoc, playbooks, idempotencia.
- Objetivo: baseline Linux funcional.

Fase 2 (2-4 semanas)

- Roles, handlers, templates, variables por entorno, Vault.
- Objetivo: automatización reusable y segura.

Fase 3 (1-2 meses)

- CI/CD, Molecule, estrategia de despliegue, Windows/red.
- Objetivo: operación robusta en equipos.

Fase 4 (continuo)

- AWX/AAP, compliance, catálogo interno de roles, SRE runbooks.
- Objetivo: plataforma de automatización enterprise.

26 Cheat Sheet final

```
# Inventario
ansible-inventory -i inventories/dev/hosts.ini --graph
ansible all -i inventories/dev/hosts.ini -m ping

# Playbooks
ansible-playbook -i inventories/dev/hosts.ini playbooks/site.yml
ansible-playbook -i inventories/prod/hosts.ini playbooks/site.yml --
    check --diff

# Vault
ansible-vault create group_vars/prod/vault.yml
```

```
ansible-playbook -i inventories/prod/hosts.ini playbooks/site.yml --ask-vault-pass

# Calidad
yamllint .
ansible-lint
```

Listing 21: Comandos que debes dominar

27 Glosario rápido

Término	Definición práctica
Inventory	Lista de hosts y grupos objetivo.
Playbook	Archivo YAML con automatización declarativa.
Role	Componente reusable con tareas, vars y handlers.
Collection	Paquete de contenido Ansible (módulos/plugins/roles).
Vault	Cifrado de secretos para uso seguro en playbooks.
Idempotencia	Ejecutar múltiples veces sin producir drift no deseado.
Handler	Tarea disparada por <code>notify</code> , normalmente para reinicios.
AWX/AAP	Plataforma de ejecución centralizada y gobernada de Ansible.

28 Conclusión

Ansible alcanza su nivel *master* cuando se combina:

- diseño limpio de roles y variables,
- seguridad de secretos y acceso,
- pruebas y automatización CI/CD,
- y operación con trazabilidad en producción.

Si aplicas este contenido con laboratorios y revisión técnica en equipo, tendrás dominio profesional de Ansible para entornos reales.